IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

SPECIAL SECTION ON INTERNET-OF-THINGS ATTACKS AND DEFENSES: RECENT ADVANCES AND CHALLENGES

Received October 17, 2020, accepted October 21, 2020, date of publication October 28, 2020, date of current version November 18, 2020.

Digital Object Identifier 10.1109/ACCESS.2020.3034324

Cyber Resilience in Healthcare Digital Twin on Lung Cancer

JUN ZHANG[1], LIN LI[1], GUANJUN LIN[2], DA FANG[1], YONGHANG TAI[1], AND JIECHUN HUANG[3]

[1]School of Physics and Electronic Information, Yunnan Normal University, Kunming 650000, China
[2]School of Information Engineering, Sanming University, Sanming 365004, China
[3]Department of Cardiothoracic Surgery, Huashan Hospital, Fudan University, Shanghai 200040, China

Corresponding authors: Yonghang Tai (taiyonghang@ynnu.edu.cn) and Jiechun Huang (huangjiechun@huashan.org.cn)

ABSTRACT As a key service of the future 6G network, healthcare digital twin is the virtual replica of a person, which employs Internet of Things (IoT) technologies and AI-powered models to predict the state of health and provide suggestions to a range of clinical questions. To support healthcare digital twins, the right cyber resilience technologies and policies must be applied and maintained to preserve cyber resilience. Vulnerability detection is a fundamental technology for cyber resilience in healthcare digital twins. Recently, deep learning (DL) has been applied to address the limitations of traditional machine learning in vulnerability detection. However, it is important to consider code context relationships and pay attention on the vulnerability related keywords for searching an IoT vulnerability in healthcare digital twins. Due to massive software and complexity of healthcare digital twin, a full automatic solution is really needed for assisting cyber resilience check in the real-world scenarios. This article presents a novel scheme for recognising potential vulnerable functions to support healthcare digital twins. We develop a new deep neural model to capture bi-directional context relationships among the risky code keywords. A number of well-designed experiments are carried out on a large ground truth, which consists of tens of thousands of vulnerable and non-vulnerable functions from IoT related software. The results show our new scheme outperforms the state-of-the-art DL-based methods for vulnerability detection.

INDEX TERMS Internet of Things (IoT), healthcare, digital twin, cyber resilience, lung cancer.

I. INTRODUCTION

Digital twin will be one of three important 6G services in the next decade, as stated in Samsung 6G vision [1]. With the combination and optimization of advanced technologies such as IoT, AI and analytics, digital twins are developed as the virtual replicas of physical entities, including people, devices, objects, systems, and even places. Professional people can run simulations of using digital twins before actual actions are performed on physical entities. There are a number of digital twin business applications in three sectors: manufacturing, automotive and healthcare. The explosion of IoT sensors is a key driver of digital twins. Digital twins can radically alleviate maintenance burdens because of the capability of a real-time monitoring what's happening with physical entities. For instance, Chevron saves millions of dollars in maintenance costs through applying digital twin for its oil fields and refineries. Siemens uses digital twins to model and prototype objects that have not been manufactured yet, which can reduce product defects and shorten time to market.

The associate editor coordinating the review of this manuscript and approving it for publication was Weizhi Meng.

In healthcare, a digital twin ideally can be the virtual replica of a person, which employs a life-long data record and AI-powered models to predict the state of health and provide suggestions to a range of clinical questions [2]. A healthcare digital twin can be used to predict the outcome of specific procedures such as lung cancer diagnosis. For example, GE developed a platform of healthcare digital twin that leverages a combination of discrete event and agent based methods to specifically account for the nuances of care delivery and how patients flow through a healthcare system. A healthcare digital twin can track a patient's records and crosscheck them against registered patterns in a healthcare system and analyze the disease indications in an intelligent way. It applies data mining and machine learning algorithms to produce accurate outcomes with continuous updating of data acquisition and processing capabilities [3]. A healthcare digital twin can benefit the cost savings greatly because its services produce a lifetime growth for the hospitals and organizations. It can assist in predicting emergencies like lung cancer, cardiopulmonary and respiratory arrest, thereby helping the hospitals and organizations in better precautionary maintenance while providing precision health care cost.

Cyber resilience [1], [4] is a big concern in the development and applications of healthcare digital twins. The large volume of patient data collected, transmitted and processed, which is conveyed back and forth to the healthcare digital twin, can make managing the scale of these hospital processes worried. Hospitals and organizations must ensure that the patient information is secure and trusted throughout the whole digital twin process. Therefore, cyber resilience should work as an embedded enabler of innovation through the use of cryptography, authentication and authorization, and vulnerability detection technologies. cyber resilience builds on security, policy and risk management to provide the trustworthy required for healthcare employees, partners and patients to work with a hospital at digital speed and scale. Cyber resilience is a critical enabler of trustworthy in the operation of precision medicine using healthcare digital twins. To support wide applications of healthcare digital twins, the right cyber resilience technologies and policies must be applied and maintained to preserve cyber trustworthy.

Vulnerability detection is a fundamental technology for cyber resilience in healthcare digital twins and the Internet of Things (IoT) [5]. Exploitable vulnerabilities in the medicine IoT related software can pose critical security threats to the digital twin systems, and impact on millions of users [6]. For example, Heartbleed was an open-source software's big security issue. A small OpenSSL programming mistake opened a security hole in a software that affected hundreds of millions of websites. Traditional vulnerability detection techniques have two basic categories, static and dynamic [7]. Static techniques, such as rule/template-based detection, code clone detection, and symbolic execution, often produce many false positives. Dynamic techniques such as fuzz testing and taint analysis often have the problem of low code coverage.

Machine learning (ML) has been applied in software vulnerability detection, which is complementary to traditional static and dynamic techniques in MIoT security [8]. The ML techniques have the capability of learning hidden vulnerable code patterns, with a great potential of detecting zero-day vulnerabilities and the variants [9]. Existing ML-based vulnerability detection methods use human-readable code features and traditional ML models, including support vector machine (SVM) and decision tree [10]. Some features of source code such as header files, function calls and software complexity metrics have been proposed to recognise possibly vulnerable files or code snippets. The programming-related data such as developer activities and code commits, which can be obtained from the version control systems, were also collected and used in vulnerability analysis [11]. Feature engineering is a key component in the existing ML-based methods for vulnerability detection, which depends on programming experience, level of vulnerability expertise, and depth of IoT security domain knowledge [12]. The emerging deep learning techniques demonstrate excellent potential to address the limitations of ML-based vulnerability detection [13], [14]. It is a new research trend of using deep neural models to build an effective vulnerability detector in cybersecurity [15].

Some early works apply fully-connected neural networks (FCNs) to learn high-level discriminative features from the low-level features that are manually crafted [16]. Considering software code is sequentially dependent data, CNNs are proposed to learn representations from code context of small sizes [17]. RNNs are introduced to model dependencies in code sequences and learn code vulnerability patterns [18]. However, the nature of a vulnerable function is about bi-directional code relationships, and it only relies on several vulnerability-related keywords. In the real world, cyber professionals need an end-to-end solution to help recognise potential vulnerable functions for manual check. These motivate our work presented in this article.

There are three major contributions of this article. A novel end-to-end scheme is proposed for cyber resilience, which can recognise potential vulnerable functions in software projects for healthcare digital twins. A new deep code attention technique is developed to explore context code relationships among vulnerability related keywords. A number of experiments are carried out to evaluate the proposed scheme on two ground-truth datasets which includes tens of thousands of vulnerable and non-vulnerable functions. The results show the new scheme outperforms the state-of-the-art DL-based methods.

The rest of the paper is organized as follows. Section II introduces the background and related work. Section III presents a framework of healthcare digital twin for the lung cancer diagnosis. Section IV presents a new scheme of vulnerable detection for the healthcare digital twin. Section V reports a set of well designed experiments along with the empirical results. Section VI concludes this article.

II. BACKGROUND AND RELATED WORK

A. ML-BASED LUNG CANCER PREDICTION

The traditional method for lung cancer prediction is based on the diagnostic criteria including the clinical symptoms, physical signs, and unknown causes of patients. Furthermore, some research suggest to combine the clinical symptoms and conventional diagnosis such as electrocardiogram, conventional d-dimer, spiral CT pulmonary, and pulmonary angiography, to predict the lung cancer with pulmonary embolism (PE) [19]. In real-world scenarios, there needs an intermediary translation, i.e., a doctor has to quickly establish an accurate connection between the patient's health status and the actual condition. The conventional pathological diagnosis may lead the tumor cell to metastasize by blood and lymphatic vessel. Machine learning are likely to assist the oncological surgeons in digging the deep relationships between the data-based medical examinations and the individualized treatment. Most of existing research apply the general chi-square test or linear regression model to analyse the risk factors of lung cancer complicated with PE were conducted [20]. Since the datasets are lack of the regional specificity, the col-linearity among variables can not be solved by using the regression model alone and the models are not applicable to the clinical decision-making [21].

Machine learning have been employed to predict lung cancer in existing studies. A framework is proposed to assess the risk level of the incidence for PE and deep vein thrombosis using an artificial neural network [22]. In this work, 31 risk factors are used to determine 294 inpatients and optimize the artificial neural network model to predict the patients' condition, which can reduce perfusion scanning rate. After that, a decision analysis model is developed for CT imaging of adult PE patients [23], which transforms the original EMR data as a timeline into a feature vector to enhance the CT's contribution for PE patients. The network is used to predict the condition of patients to determine whether the patients need to exam the CT necessarily. Moreover, Several machine learning methods including kNN, naive abyesian and decision tree are compared in the diagnosis of lung cancer with PE on a dataset of 201 patients. The decision tree and Bayesian method are also selected to analyse the possibility of PE with limited study tests and for the patients' difficulty diagnosed [19]. Recently, chest X-ray is considered as the best way to diagnose pneumonia, which plays an essential role in clinical nursing and epidemiological research. A PA model named CheXnet, which is a 121-layer convolutional neural network, is developed for predicting the pneumonia [24]. The empirical study demonstrates that the CheXnet outperforms the radiologist in terms of sensitivity and specificity in pneumonia detection tasks [24].

B. HEALTHCARE DIGITAL TWIN AND ITS SECURITY

The number of intelligent Medical Internet of Things (MIoT) and healthcare digital twins deployed has been constantly increasing. The estimated market value will continually increase to $75.44 billion in next decade [25]. There is a huge potential for the application of IoTs in health industry, but practical constraints must be taken into consideration. An approach is presented to combine vending machines with the IoT technology to facilitate a healthy lifestyle [26]. However, cyber-attacks are not new to MIoT, which will lead to terrible consequences to healthcare digital wins. With the widespread application of IoT devices, cyber-attacks are also improving, posing a severer threat to the secure operation not only to IoT devices but also to the entire cyber space [10].

With an increasing number of MIoT related cyber incidents being reported, experts and researchers from MIoT industry and academia have been working to design secure systems and solutions to combat the attacks of various types [6], [14]. Many researchers have devoted extensive efforts to ensuring MIoTs security and privacy, providing practical guidance for MIoT security. Both opportunities and possible threats that IoT face in two important application scenarios of home and hospital are studied in [27]. An extensive survey is provided in [28], which presents the classification of MIoT attacks from perspectives of MIoT security research, threats, and open issues. A new method is developed in [29], which uses utilized Unmanned aerial vehicles (UAV) to launch an attack in a simulated smart hospital environment and compromise a small collection of wearable healthcare sensors. A new deep learning approach, DFEL, is proposed for real-time cyber-attack detection in the IoT environment, and presented the robustness of high accuracy and significant time-saving [30].

The IoT application has been widely used in the medical industry. In recent years, it has become widespread to combine Virtual Reality (VR) technology with medical-related majors. To make the operation of the entire medical platform more transparent, we adopted the combination of VR technology and MIoT to correctly reproduce the prediction process of lung cancer complicated with pulmonary embolism through the medical platform. The integration of the Internet of Things and VR technology in the education field can enable learners to combine their conceptual learning with practical experience in a novel way [31]. A large amount of data generated by sensor devices such as road information, air conditions, and traffic control in the city are imported into the virtual environment, and the combination of VR and the IoT is used to create a virtual smart city. Coogan at al. uses unity software, combined with a brain-computer interface to control the VR environment and MIoT devices [32].

C. DL-BASED VULNERABILITY DETECTION

Early studies applied the fully-connected network (FCN) for learning code features for vulnerability detection. This kind of network can be regarded as a highly nonlinear classifier for learning hidden and potentially complex fragile patterns. In the case of a large data set, compared with the traditional ML algorithms, FCN has the potential to build a complex model with the good capability of capturing nonlinear characteristics. This potential prompts researchers to use it to analyse source code and recognise the patterns of software vulnerability. Input structure independent is another advantage of FCN, i.e., the neural network can take various forms of input data such as images and sequences [33]. It enables researchers to use different handcrafted features and explore different information to discover potential vulnerabilities in source code. Shar and Tan [34] adopted the FCN to detect SQL injection and cross-site scripting vulnerabilities on PHP applications. The authors extracted features from control flow graphs (CFGs) and data dependency graphs (DDGs) and applied the FCN to build up a detection model in the feature space. Their research shows that FCN is superior to other two conventional ML classifiers, C.45 and Naive Bayes, on their data sets. Grieco *et al.* [35] used bug trackers to collect vulnerability codes from the Debian operating system and employ an FCN to learn static and dynamic characteristics for detecting memory corruption vulnerabilities. Dong *et al.* [36] applied smali2vec to capture some features of the Smali (apk decompile file) in the apk. Then they adopted the FCN into binary-level vulnerabilities detection in Android applications using the features extracted from the Dalvik instructions. Their results suggested that the FCN-based model outperformed traditional machine learning methods such as support vector machines. Peng *et al.* [37] used the N-gram model to extract the frequency-based features from the surface

text of Java source code. The trained FCN model identified vulnerable files/classes in Android applications.

However, most of existing FCN-based methods take manually crated features as input to build vulnerability detection models. These research usually require different feature engineering methods for different problems. The detection models' performance relies on the effectiveness of the features and the knowledge of feature extraction. In addition, FCN is not originally designed for processing sequentially dependent data, such as the program code in the context of software vulnerability research. Software vulnerabilities are usually caused by the joint action of context in the control flow of the code [38]. For example, in some 'Use After Free' vulnerabilities, vulnerable code snippets may use variables that have released memory in previous code. Hence, the network structures designed for processing context-aware information were utilized by researchers for vulnerability detection. A common assumption is that software code contains semantics and syntax similar to natural languages. Some ideas and techniques developed for natural language processing (NLP) have been used to learn the features of code vulnerabilities. Convolution neural network (CNN) aims to deal with structured spatial data [33]. CNN can learn features from semantically similar pixels for image classification. This capability can also be used for NLP tasks. For example, in the text classification task, the CNN applied to the context window (containing a small number of word embedding) can capture the word features in the context window, thus achieving the purpose of capturing the text context features. As a result, this prompts researchers to apply CNN to learn the context-aware code semantics in the scenario of software vulnerability detection. Researchers have achieved some success in using this approach. For example, CNN was proposed for building a vulnerability classifier over the features of assembly instructions in [39] and used vulnerability representation learning in [40]. The experimental studies demonstrated that a random forest classifier trained by the feature sets produced by a CNN could lead to better performance compared to that of using a CNN as a classifier.

In contrast to feed-forward networks like FCN or CNN, RNN is naturally designed to handle the sequential data and model the data dependencies. RNN has been a choice for researchers to conduct vulnerable code pattern recognition. Wu *et al.* [18] studied the performance of LSTM, a variant of RNN, in predicting vulnerabilities in binary programs. Their results show that the LSTM model's ability to discover vulnerabilities is better than that of traditional multi-layer perceptrons. However, a single LSTM can only consider code relationship in one direction. In order to make up for this limitation, Li *et al.* [41] applied the bidirectional LSTM (Bi-LSTM) network for detecting the vulnerabilities of buffer error and resources management error based on ''code gadget'' that is a program representation depicting variable flows. The addition of Bi-LSTM allows their model to capture contextual information in two directions, enhancing the model's ability to acquire vulnerability features. This is critical to understanding the semantics of many types of vulnerabilities, e.g., buffer overflow vulnerabilities. These vulnerabilities are associated with code contexts that contain multiple continuous or interrupted code fragments. There is also a line of studies that extracted the abstract syntax trees (ASTs) from source code functions and used ASTs as inputs to Bi-LSTM network for learning feature representations [11], [42]. A systematic approach was proposed by Li *et al.* [43] to apply two RNN variants (i.e., a Bi-LSTM and a Bi-GRU) for learning syntactic and semantic characteristics of code from both AST and control-flow graph (CFG). Lin *et al.* [44] proposed a multi-domain approach of utilizing two Bi-LSTM networks for learning feature representations from two different data sets, which contain code samples from the open-source software projects and the synthetic test cases.

III. DIGITAL TWIN FOR LUNG CANCER DIAGNOSIS

With the help of advanced sensors, AI, and communication technologies in the 5G/6G era, healthcare sector has the potential to replicate the physical body in a virtual world, i.e., healthcare digital twin. In future, doctors can explore and monitor the patient body and detect problems remotely through digital twins. Fig. 1 shows a healthcare digital twin designed for the diagnosis of lung cancer with PE, which links the status of real-world patient, DL driven prediction and the physics-based rendering for surgery. Data will enable the interaction between physical body and digital replica.

A. REAL-WORLD PATIENT DATA

The clinical dataset consists of the experimental group (lung cancer complicated with PE patients) and the control group (lung cancer patients), which was collected in three different hospitals from Kunming, Qujing, and Chongqing. Meanwhile relevant clinical data were summarized in terms of the risk factors for lung cancer with PE through a multicenter retrospective study and meta-analysis, combining epidemiology with evidence-based medicine. The features of the age, gender, risk factors, d-dimer level, electrocardiogram (ECG), TNM stage, leukocyte, pathological type, pathological location, and thoracic CT findings of the patients were recorded. The patient data were uploaded to the hospital system by the electronic medical record (EMR). Since some of the patients' data were recorded by hand, which may cause various errors in the data. Besides, some patients may have their first diagnosis and treatment in an external hospital and their second visit in this hospital, which may also result in the lack of data. For example, for patients who have had surgery, the time of operation was not mentioned in the medical record. Surgery is the most crucial treatment for lung cancer patients. After lung cancer surgery, patients may develop arrhythmias, including atrial fibrillation, premature beats, and atrioventricular block, due to the trauma of thoracotomy, the direct stimulation of operation, and the imbalance of vegetative nerve function.

B. PHYSICS-BASED RENDERING FOR SURGERY

There are two pipelines for rendering in our navigational surgical simulator. The visual rendering pipeline is based on

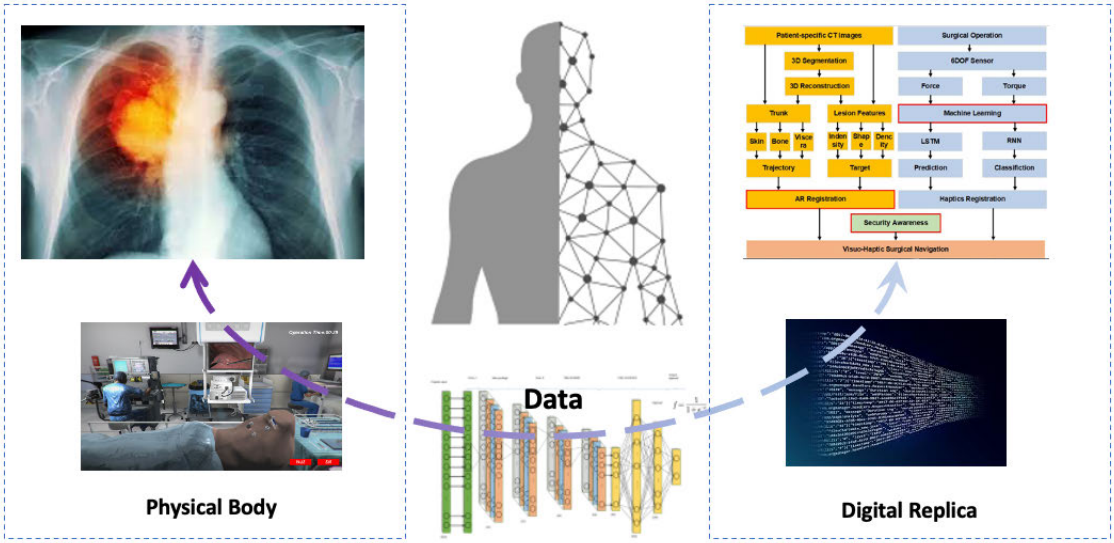


FIGURE 1. A healthcare digital twin for lung cancer.

the patient-specific CT image reconstruct mesh. To present the lifelike rendering model of visualization, we chose the physics-based rendering (PBR) algorithm during the simulation. Different from the empirical bidirectional reflectance distribution functions (BRDF), the microfacet model with more physics process could be adjusted during the visual rendering. The model has several parts, the diffuse reflection part, the specular reflection part, the visual direction, the normal of the mesh, the incident direction, the halfway vector, the hemisphere, and the material roughness. Because of the different Fresnel term, the diffuse and specular reflection have different form, the rendering model of Disney and the specific glistening effects of human organ as shown in the empirical study. We have done some modification on the Disney model to make some improvement.

Unity Software is a multi-platform integrated game development tool that allows players to easily create interactive content such as 3D video games, architectural visualization, real-time 3D animation, etc. This is a fully integrated professional game engine. The core code of the Unity engine itself is written in the underlying language C/C++. The image, sound, and physics engines are all compiled in C++. The dynamic link library DLL file encapsulates a series of methods and classes. C#, Python, and other programs call corresponding methods and classes through DLL files to build the game flexibly and superior performance. Unity can run across platforms, such as Android, ios, PC, Web, and other platforms. This article is for the Android platform. Unity will publish the APK file of the VR project to the Android device and then display it through the headset. Unity will publish the APK file of the VR project to the VR headset and display this scene. The VR headset USES Pico G2 (Beijing Bird-Watch Technology Co., LTD.) mobile VR headset, which has a field of view of 101°, refresh rate of 90HZ, and resolution of 3K, providing the wearer with immersive medical VR application scenes.

C. DL DRIVEN PREDICTION

We develop a customised CNN with data imputation to perform the prediction of pulmonary embolism in the lung cancer. The nearest neighbour (NN) method is applied to deal with missing data before deep learning. The purpose of NN interpolation is to find the nearest neighbor of the missing data from all complete instances in the dataset. The missing value is filled by the corresponding value of the nearest neighbour. The NN method has demonstrated effectiveness and simpleness in the area of recommendation. The customised neural network has two convolution layers, two full-connection layers and a sigmoid activation layer. Every convolution layer equips with a kernel with the size of 3, that is followed by a LeakyReLU activation layer and a max pool layer to down sample. The convolution layer can learn high level features from the input data. There are two full-connection layers. One layer has the input size of 320 and the output size of 120. The other layer has the input size of 120 and the output size of 2. A LeakyReLU activation layer is added between two full-connection layers. A sigmoid neuron is used for classification.

IV. VULNERABILITY DETECTION FOR HEALTHCARE DIGITAL TWINS

To capture an adequate representation of context dependencies in code and highlight crucial information, we propose a new solution utilizing the Bi-LSTM network with a self-attention mechanism. The network structure of the new solution is shown in Fig. 2. First, we apply a word embedding model to obtain the word vector to retain the meaning of each fragment in the code. Then we use the Bi-LSTM to improve the representation of the code snippet further. The Bi-LSTM network can better capture the context-dependency in the code through the forward and reverse LSTM, and obtain a vector representation with richer semantic features. Finally, by using the self-attention mechanism, we can grasp

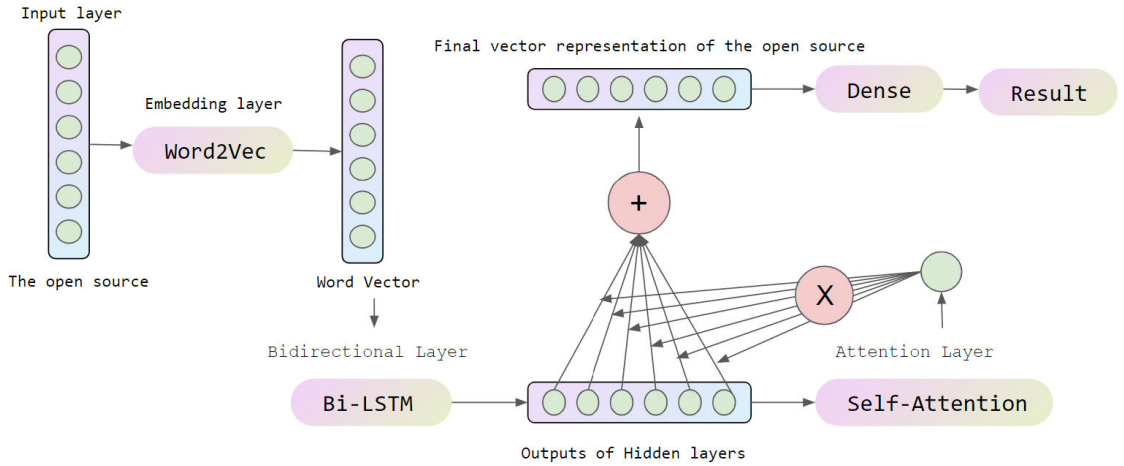


FIGURE 2. The new vulnerability attention scheme.

the key information in the code, that is, the parts that have a more significant impact on the vulnerability. This solution has two important advantages, optimizing the accuracy of vulnerability detection through better code representation and interpretable capability provided by the attention mechanism.

A. PROCESSING OF INPUT CODE DATA

We extracted functions from 9 open-source projects provided by Lin *et al.* [44] and the Software Assurance Reference Dataset (SARD) project [45] dataset. By observing the code file, we notice that there are many comments in the code, and the model may learn the features of these comments. In a real-world environment, comments do not affect the functionality of the code. The model learns the annotations' features, which may lead to finding incorrect information about the vulnerability. Therefore, during code preprocessing, we remove the annotations to avoid the model learning useless features, leaving only the code's contents. Then we separate the contents of the code according to spaces and operators (+, −, *, / and alike). This way, we can extract the keywords, variable names, function names, operators, and many more from the code. As shown in Fig. 3, we convert an example function into a sequence, with each part being equivalent to a word in natural language. Each word we call a token, which is the smallest component of the task of processing the sequence.

In traditional natural language processing tasks, the token may be converted to lowercase, or the stem of a word may be extracted. Nevertheless, in code representation, changing a character in a variable can cause the variable to have a completely different meaning. For example, the case-sensitive "a" and "A" indicate two completely different variables that point to different memory addresses. Thus we do not deal with each token individually. As mentioned in [46], natural language and software codeshare some characteristics, but they have different natures. We treat the processed code snippet sequence as a text sequence. Subsequent processing considers natural language techniques.

In the programming code, the semantics of a word is often independent of the word itself, and highly related

```
1. //before code processing
2. static void linkexten(struct ael *exten, struct ael *add)
3. {
4.     /* this will reverse the order. Big deal. */
5.     add->next_exten = exten->next_exten;
6.     exten->next_exten = add;
7. }
8.
9. //after code processing
10. ['static','void','linkexten','(','struct','ael','*','exte
    n',',','struct','ael','*','add',')','{','/','*','this','w
    ill','reverse','the','order.','Big','deal.','*','/','ad
    d','-','>','next_exten','=','exten','-','>','next_exte
    n',';','exten','-','>','next_exten','=','add',';','}']
```

FIGURE 3. An example of code processing.

to its position in the code flow. For example, we have two sequences of codes, ["c","=", "5", "+", "3"] and ["d","=", "5", "+", "3"], where the variables "c" and "d" are the same in terms of the value and the context. The semantics of what they are expressing may be similar. When mapping the two variables into a new vector space, their positions in vector space are supposed to be close to each other.

In this article, the CBOW model of word2vec is applied to capture this kind of semantics of the variables. Since the CBOW model generates the code vectors of the target code words through the context in the corpus, the different corpus will have different contexts for the target code words. To make our code embedding model represent source code better, we train the respective code embedding models using nine open-source projects and the SARD dataset. This setup allows us to acquire the ability to capture the semantic information that the stream of code in the dataset is attempting to represent. The training of the CBOW model is unsupervised. We do not use vulnerability labels and only consider code understanding and representation at this stage. In the output code representation, the complete token sequence vector is for the function's full semantics. In the programming code, the interaction between the individual code fragments in a code sequence may cause a vulnerability. This problem can also be handled by the proposed code embedding method.

Word2vec is a word embedding model by which we can transform each word in the text into a dense vector of fixed size [47]. We treat the source code of the software as text and use the one-hot encoding. Suppose one hot encoding sequence for the target code word is $Word_i = [x_1, x_2, x_3, \ldots, x_c]$, C is the total number of contexts for the target word, every X is one-hot encoding. The target code word is y_i as the label to be predicted. We can construct a neural network as follows. The weight between the input layer and the hidden layer can be expressed by a $V \times N$ matrix W. Each line of W is an N-dimensional code vector v_w, representing the corresponding code words in the input layer. Line i of W is denoted by v_w^T. Given a context $x_1, x_2, x_3, \ldots, x_c$, we can get the following formula:

$$\begin{aligned} h &= \frac{1}{C} W^T (x_1 + x_2 + x_3 + \cdots + x_c) \\ &= \frac{1}{C} (v_{w1} + v_{w2} + v_{w3} + \cdots + v_{wc})^T \end{aligned} \tag{1}$$

where w_1, w_2, w_3, …, w_c is the contextual word, and the v_w is the input code vector of the code word w_c. There is a different weight matrix between the hidden layer and the output layer. $W' = \{w'_{ij}\}$, This is an $N \times V$ matrix. Using these weights, we can calculate the score u_j for each code word in the vocabulary that consists of V words. We note that v'_{w_j} is the j-th column of matrix W'.

$$u_j = v'^T_{w_j} h \tag{2}$$

The loss function is set as follows:

$$\begin{aligned} E &= -logp(w_{target} | w_1, w_2, w_3, \ldots, w_c), \\ &= -u_{j*} + log \sum_{j'=1}^{V} exp(u_{j'}), \\ &= -v'^T_{wo} \cdot h + log \sum_{j'=1}^{V} exp(-v'^T_{wj} \cdot h) \end{aligned} \tag{3}$$

where $j*$ is the actual output of the cod words in the output layer. We use the following expression to update the weight of the hidden layer to the output layer:

$$v'^{(new)}_{w_j} = v'^{(old)}_{w_j} - \eta \cdot \frac{\partial E}{\partial W'}, \qquad j = 1, 2, \ldots, V \tag{4}$$

For each code function in the training set, we will apply this equation to every element of the weight matrix between the hidden and output layers. We use the following equation to update the weight between the input layer and the hidden layer:

$$v'^{(new)}_{w_j} = v'^{(old)}_{w_j} - \frac{1}{C} \cdot \eta \cdot \frac{\partial E}{\partial W'}, \qquad C = 1, 2, \ldots, C \tag{5}$$

where v'_{w_j} is the input code vector of the jth code word in the input function, and η is the learning rate. After training the model, we can get the vectorization representation of a code word with the following formula:

$$Word_i \, Embedding = W^T x_1 \tag{6}$$

B. CONTEXT CODE RELATIONSHIP DISCOVERY

The similarity between code sequences and "sentences" in natural language prompted us to apply RNN into the vulnerable function detection. Since the sequence of the contained code vector reflects structural information, modifying the sequence of any item will change the structure and semantics. For example, a code sequence [void, main, …] represents a void type returned by the main function. If the order of "void and "main" is switched, the code sequence becomes [main, void, …], and it is meaningless. Therefore, RNN is used to discover the structural characteristics of vulnerable functions.

Normally, the pattern of vulnerable code in software functions is correlated with multi-line code. As we map functions to a vector space, the vulnerability pattern may be related to multiple elements of a code vector. A standard RNN is capable of dealing with short-term dependency relationships but does not perform well with long-term dependencies. In our scheme, RNNs with LSTM units are used to capture long-term dependencies and learn high-level representations of vulnerable functions. A single direction LSTM can only capture dependencies in one direction. For instance, a sequence of assignment statements ["A", "=", "B"]. We assume that the LSTM direction is left-to-right. When "B" enters the LSTM unit, the LSTM unit will know that the information of "B" is assigned to "A". However, when the LSTM unit reads "A" initially, it does not get any information of "B", which is not the result we want. Hence, we use a bi-directional LSTM to capture dependencies in both directions, so that both "A" and "B" can acquire information about each other.

LSTM is a variant of RNN. It adds a gate mechanism and a memory cell to the RNN, capturing the long-term dependence of the input sequence. It can solve the problem of the gradient vanishing and exploding of the gradient of RNN [48]. Assuming that t is the current state and $t-1$ is the previous state, the following transfer equation can represent a single LSTM.

$$\begin{aligned} i_t &= \sigma(W^i x_t + U^i h_{t-1} + b^i) \\ f_t &= \sigma(W^f x_t + U^f h_{t-1} + b^f) \\ o_t &= \sigma(W^o x_t + U^o h_{t-1} + b^o) \\ u_t &= tanh(W^u x_t + U^u h_{t-1} + b^u) \\ c_t &= i_t \odot u_t + f_t \odot c_(t-1) \\ h_t &= oi_t \odot tanh(c_t) \end{aligned} \tag{7}$$

where i_t, f_t, o_t represents three gate units, the input gate, the forget gate, and the output gate, respectively. W^i, W^f, W^o correspond to their weight matrices, and b^i, b^f, b^o are the biases of their gates. x_t is a word vector of the current state. h_{t-1} is the output of the previous state. U is the matrix of weights corresponding to h_{t-1}. The symbols *sigma* and *tanh* denote different activation functions, S-Functions and hyperbolic tangent functions, respectively. $\odot$ is the Hadamard Product, which is the multiplication of the corresponding elements in the operation matrix.

Vulnerabilities are often caused by a combination of the context of vulnerability fragments. LSTM can only capture the previous information, not the information behind it. To enhance the ability to capture contextual information, we apply the bidirectional form of the LSTM, which use two LSTM networks of two opposite directions, to capture the forward and reverse code relationship. The final hidden layer output of Bi-LSTM is calculated below:

$$h_t = [\overrightarrow{h_t}, \overleftarrow{h_t}], \tag{8}$$

where $\overrightarrow{h_t}$ and $\overleftarrow{h_t}$ represent the output of the hidden layer in forward and reverse LSTM.

C. HIGHLIGHTING VULNERABLE KEYWORDS

In real vulnerable code, not all code snippets are associated with the vulnerability, and the existence of the vulnerable code statements may be intermittent. For all sequences in Bi-LSTM, each token's weight parameters are the same, which makes the model equipped with only the Bi-LSTM layer without the focus on the key tokens that are relevant to vulnerabilities. Implementing a self-attention mechanism can enable the model to focus on the critical tokens associated with the vulnerability while paying less attention to irrelevant tokens. For example, In Fig. 8, not all code blocks contribute equally to the vulnerability. In particular, the sequence [''preempt_enable'', ''('', '')'', '';''] are irrelevant to the vulnerability. The special vulnerability is mainly caused by the call to the ''_raw_read_unlock'' function. Ideally, the detection model should focus more on whether the ''_raw_read_unlock'' function is called correctly, rather than pay equal attention to the code statements that are less likely to lead to the vulnerability.

Moreover, the separate attention layer has its problems because it does not capture the order of relationships in a sequence. For example, in the sequence [''A'', ''B'', ''C''], ''A'' precedes ''B''. This order is important for vulnerability detection because there is a strict order of execution in the code. The incorrect order can lead to vulnerabilities. Considering Bi-LSTM can learn the sequential relationships of code words, we attach the attention layer to the Bi-LSTM layer. Our scheme allows the attention layer to capture the sequence order and focus more on the critical features of the vulnerability. It assigns different weights to different code blocks. Consequently, our new scheme can achieve a better understanding of the key code words for vulnerable function detection.

The essence of the attention mechanism comes from the human visual attention mechanism [49]. When people perceive something visually, they do not always see a scene from beginning to end, but rather on an as-needed basis. Observation pays attention to a particular part [50]. When people find that a scene often has something they want to observe in a certain part, they learn to reappear in the future, focusing the attention on that part when similar scenes occur. Similarly, in the open-source code, not all code fragments contribute equally to the vulnerability fragments. Some code snippets may be the root cause of the vulnerability. For example, in Use After Free (UAF) vulnerability, the root cause can be the use of a variable that has previously been freed from memory. The code snippet that frees the memory variable would need to receive more attention to see if it continues to be used in subsequent code. Based on this observation, we propose using a self-attention mechanism to assign different weights to different code fragments to indicate the difference in their contributions to the vulnerability fragments.

In the traditional attention mechanism, the attention function is described as a mapping from query to (key-value) pairs [51]. In other words, the value corresponding to the key is obtained by calculating the similarity between the query and the key. The weight is usually calculated using the following formula:

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V. \tag{9}$$

Query, key, and value maintain three weight matrices W_q, W_k, W_v, respectively. Q, K in the above formula is obtained by multiplying the input word vector by the corresponding weight matrix, respectively. d_k is used to adjust the score so that the internal product is not too large. Through the above calculation, we can obtain an attention value of the query. In self-attention, the size of Q is equal to K and V. In such a situation, when we enter a code sequence, every coed word in the sequence needs to compute an attention vector with all the other words in the sequence. So, no matter how long the distance between each other, the maximum path length is 1. This makes our model be able to better capture the longer distance dependencies in the code function [52]. Due to the introduction of the attention mechanism, we can know which code word can have a greater impact in terms of the model's final decision on vulnerability, which improves the interpretability of the model [53].

In our scheme, the output of Bi-LSTM is an h_t with forward and reverse hidden layers. The Self-Attention layer can be shown in the following equation:

$$\begin{aligned} e_t &= \sigma(w_a h_t + b_a) \\ a_t &= softmax(e_t) \\ l_t &= \sum a_t h_t \end{aligned} \tag{10}$$

We pass the output of each state in Bi-LSTM h_t through a perceptron and then perform softmax activation to get the weights a_t. Finally, the weights a_t and h_t are optimised and summed to obtain the final vector representation of the code word.

V. EXPERIMENTS AND RESULTS

This section reports the experiments and results for evaluating the proposed scheme for vulnerable function detection.

A. DATA AND METRICS

1) TWO IOT RELATED GROUND-TRUTH DATASETS

The SARD contains synthetic test cases which are designed as proof-of-concept vulnerabilities and patches. Many of

TABLE 1. The data sets used in our experiments.

Dataset	Software project	# of Vulnerable functions	# of Non-vulnerable functions
9 open-source project	Asterisk	94	752
	FFmpeg	249	1,992
	Httpd	57	456
	LibPNG	45	360
	LibTIFF	123	738
	OpenSSL	159	1,272
	Pidgin	29	232
	VLC	44	352
	Xen	670	5,360
The SARD project	Synthetic C code samples	27,367	27,367

the vulnerabilities are well-known in IoT security area. We believe that these artificially constructed test cases exhibit ''basic patterns'' of vulnerabilities. In this article, we collected a large number of the C test cases from the SARD project and extracted 27,367 vulnerable functions and their fixed version (i.e., non-vulnerable functions) from the test cases. There are a large number of functions with the same name in the SARD dataset. For example, vulnerable functions are named with the phrase ''bad'' or ''badSink'', while non-vulnerable functions are named with names containing the words ''good'' or ''goodSink''. In order to prevent our model from learning the characteristics of the vulnerability through function names, we change all function names into the same string 'function'.

The open-source project dataset we used is created by Lin *et al.* [44]. This dataset consists of vulnerable and non-vulnerable functions from 9 IoT related software projects written in C programming language. The 9 projects are Asterisk, FFmpeg, Httpd, LibPNG, LibTIFF, OpenSSL, Pidgin, VLC, and Xen. The labels are manually created according to the records in the Common Vulnerabilities and Exposures (CVE)[1] and the National Vulnerability Database (NVD),[2] which are publicly accessible vulnerability data repositories. This dataset consists of 1,470 real-world vulnerable and 11,512 non-vulnerable functions. The detail information is shown in Table 1. This vulnerability data made it possible to train classifiers for vulnerability detection at the functional level, which provides a more granular level of detection than the file level.

2) PERFORMANCE METRICS

In our experiments, we used 9 open-source project datasets and the SARD dataset to train the word vector of the code using Word2Vec with the Continuous Bag-of-Words (CBOW) model. Each word vector was set to the length of 100 dimensions. We use Keras [54] with Tensorflow [33] backend to build Bi-LSTM networks. In our experiments, the input sequence length is set to 1000, and the number of LSTM units is set to 64. Then we use ''SeqSelfAttention'' from Keras as the attention layer in our model to capture key information in the code.

[1] https://cve.mitre.org/
[2] https://nvd.nist.gov/

The performance metrics we use are precision, recall, F score, top-k precision, top-k recall, and Roc curve. Precision displays the probability that the functions correctly exist in the prediction list. It reflects the proportion of the correct predictions in all code functions. Recall shows the percentage of correct predictions for all functions in the vulnerability class. Although the precision and the recall can be used as systematic evaluation indicators, their values could contradict each other. F1-Measure resolves this contradiction through the weighted harmonic average of the precision and the recall.

Top-k precision and Top-k recall: They represent the first k precision (denoted as P@K) and the first k recall (denoted as R@K), respectively. These metrics are often used in the first k documents related to information retrieval [55]. $TP@K$ refers to the number of actual vulnerabilities in the first k functions that are most likely to be vulnerabilities. $FP@k$ and $FN@k$ represent the number of False Positive and False Negative in k samples.

ROC curve: The samples will be ranked based on the model's predictions, and the samples will be predicted individually in that order. Each time, the values of two important quantities — True positive rate and False positive rate are plotted with them as horizontal and vertical coordinates, respectively.

B. COMPARISON WITH THE STATE-OF-THE-ART METHODS

1) METHODS FOR COMPARISON

In the experiments, we implemented five methods for performance evaluation. Deep vulnerability recognition (DeepVR) refers to the implementation of our method proposed in this article. LSTM refers to the implementation of a Bi-LSTM without the self-attention mechanism. The higher performance is highlighted in bold fonts. In this article, we choose the methods presented in [11], the method presented in [44] and Flawfinder [56] for the comparison study. Lin *et al.* [11] used the abstract syntax tree (AST) for extracting code representations and applied a Bi-LSTM network to create a classifier. Lin *et al.* [44] trained two identical Bi-LSTM networks for learning high-level representations from two different types of datasets. Then, they combined the representations learned from two datasets and used random forests to generate the final predictions. Flawfinder is a well-known open-source tool for static code analysis. It equips with a vulnerability-feature database, allowing quick examination of C/C++ source code for possible security vulnerabilities [56]. Both the method in [11] and the one in [44] are proposed for function-level vulnerability detection. Flawfinder has been used as a benchmark by many studies, such as [57] and [58]. Therefore, we apply these methods for benchmarking.

2) RESULTS FOR DIFFERENT METRICS

To assess the impact of different datasets on the model, we tested DeepVR and LSTM on the open-source dataset and the SARD dataset. Tables 2 and 3 show the performance of the two models on the open-source and SARD

TABLE 2. Our DeepVR method vs Bi-LSTM network and Flawfinder, in the open-source dataset.

Vulnerable class	Precision	Recall	F1 Score
DeepVR	**0.79**	**0.77**	**0.78**
LSTM	0.61	0.63	0.62
Flawfinder	0.30	0.28	0.29

TABLE 3. Our DeepVR method vs Bi-LSTM network and Flawfinder, in the SARD dataset.

Vulnerable class	Precision	Recall	F1 Score
DeepVR	**0.99**	0.97	**0.98**
LSTM	0.92	**1**	0.96
Flawfinder	0.59	0.52	0.56

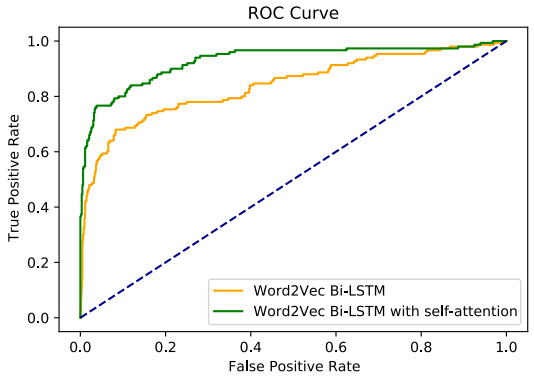


FIGURE 4. The ROC result of comparing our DeepVR method with the Bi-LSTM network, in the open-source dataset.

datasets, respectively. From Table 2, we can see that in the open-source dataset, DeepVR performs significantly better than LSTM in all three evaluation metrics, precision, recall, and F1 score. The difference can be from 14% to 18%. As shown in Table 3, in the SARD dataset, DeepVR's precision is higher than those of LSTM. The recall and F1 scores are close to each other. We observe that LSTM incorrectly identifies more samples as vulnerable, leading to a decrease in its precision. Compared with the performance on the open-source project's dataset, DeepVR and LSTM perform significantly better on the SARD dataset. The SARD dataset contains artificially constructed samples whose patterns are relatively simple and less diverse in contrast to the real-world samples from the open-source project datasets, which may make it easier for DeepVR and LSTM to capture. In two datasets, DeepVR and LSTM outperform Flawfinder dramatically.

Fig. 4 shows the Receiver Operating Characteristic (ROC) curves, which presents a different perspective of the performance of DeepVR and LSTM. The green and orange curves indicate the DeepVR and LSTM, respectively. The horizontal coordinate shows the false positive rate, which is the proportion of all good cases predicted as vulnerability. The vertical coordinate denotes the True positive rate, the proportion of vulnerable cases that are predicted as vulnerability. The dashed line in the middle reflects the performance of the stochastic model. In this figure, we can see that the area of the green curve is significantly larger than that of the yellow curve. The area of the curve denotes the value of AUC. The larger the AUC value, the more likely the current detection method will rank vulnerable samples ahead of good samples. In other words, the DeepVR model represented by the green curve will be able to detect vulnerable functions much better than LSTM.

3) COMPARISON WITH EXISTING APPROACHES

Table 4 reports the Top-k precision and recall of five competing methods in the open-source dataset. For example, LSTM correctly identified 93 vulnerabilities in the top 150 of the ranked list and obtained 62% precision and recall. Compared to LSTM, our DeepVR method can significantly improve the detection performance with the support of the attention mechanism. In particular, DeepVR can increase the performance by 15% in top-100 precision and 16% in top-150 precision.

We compared the new work with the work by Lin *et al.* [11]. For example, The performance of our DeepVR method is higher than Lin *et al.*'s by 20% in top-10 precision and 1% in the top-10 recall. This reveals that our DeepVR model can identify more vulnerabilities as early as possible. We also compared our method to the method of using combined representation [44]. In terms of top-10 precision and recall, both methods can achieve perfect performance and detect all vulnerable functions. However, as the k value increases, the method of combined representation does not work well. For example, tt can correctly identify only 30 vulnerable functions in top-150. Meanwhile, our DeepVR model gets 40 new correctly identified vulnerabilities in this interval. Our DeepVR is also compared with the traditional static method Flawfinder. As shown in Table 4, although Flawfinder correctly identifies 40 vulnerable functions in the top 150 results, it misidentifies more non-vulnerable functions as vulnerable.

C. CODE ANALYSIS AND VISUALISATION

1) CASE STUDY ON CODE RELATIONSHIP

Karpathy *et al.* [59] demonstrated that LSTM units can learn long-range dependencies of code sequences by training on the Linux kernel source code. Fig. 5 shows the visual output of the model using the "tanh" activation function, with the blue part indicating a "+1" output and the red part denoting an output of "-1". In the example, the red part identifies the range of the "if" statements, which contains multiple lines of code. In general, the bi-directional LSTM has a certain capability to capture the long-range dependency among multiple lines of code in a function that is important to detect vulnerable functions.

We build up the Word2Vec models of using the open-source dataset and the SARD dataset, respectively. Fig. 6 shows an example of a 2D vector space for code embedding, created by using the open-source dataset. In Fig. 6, one can see that the code embedding model has successfully learned some semantic meaning of the programming code. For example, operators, keywords, and variable names are mapped to separate clusters in the 2D vector space. In the lower-left corner of Fig. 6, there are the code symbol classes. Especially symbols such as "[", "]" and "{", "}" are pairs

TABLE 4. Comparison of our DeepVR with the Bi-LSTM network, Lin *et al.* [11], Lin *et al.* [44] and Flawfinder [56] in the open-source dataset.

Open-Source project	Detected vulnerable functions (Top-k precision and recall)						
	Top 10	Top 20	Top 30	Top 40	Top 50	Top 100	Top 150
DeepVR	10 (100%|6%)	20 (100%|13%)	**30 (100%|20%)**	**40 (100%|27%)**	**50 (96%|33%)**	**92 (92%|61%)**	**115 (79%|77%)**
LSTM	10 (100%|6%)	20 (100%|13%)	28 (93%|19%)	38 (95%|25%)	48 (96%|32%)	77 (77%|51%)	93 (62%|62%)
Lin et al. [11]	8 (80%|5%)	17 (85%|11%)	25 (83%|17%)	34 (88%|23%)	42 (84%|28%)	70 (7%|45%)	72 (48%|48%)
Flawfinder [56]	4 (30%|3%)	5 (25%|3%)	5 (17%|3%)	5 (13%|3%)	5 (10%|3%)	5 (5%|3%)	40 (27%|27%)
Lin et al. [44]	10 (100%|6%)	20 (100%|13%)	29 (97%|19%)	38 (95%|25%)	47 (94%|31%)	80 (80%|53%)	85 (57%|57%)

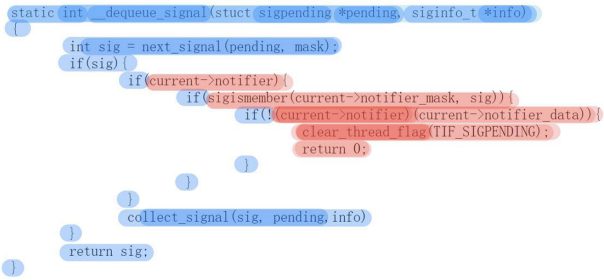
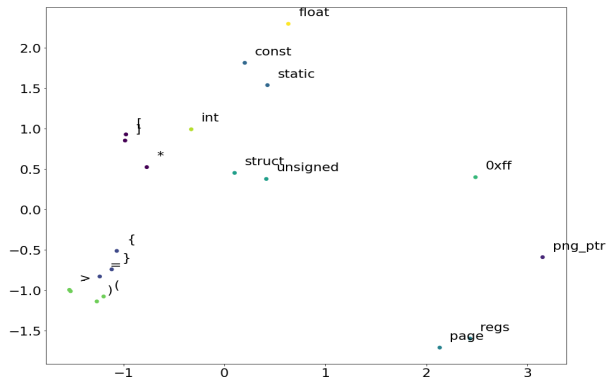


FIGURE 5. An example of interpretable dependency.

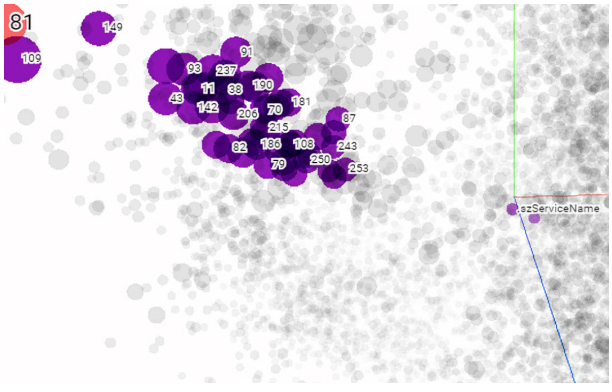
existing closely in the 2D vector space. Let us look at the keywords, "int" and "float". Although they both represent variable types in C, they are not close to each other in the 2D vector space because they have different contexts in the code. To further explore the relationship of code vectors after code embedding, we apply the principal component analysis (PCA) to reduce 100 dimensions of a code vector to 3 dimensions and map it to a 3-dimensional coordinate system, as shown in Fig. 6b. For example, in the 3D vector space, it can be seen that the corresponding numbers of code vectors are concentrated around the line emitted from the center. It shows that these code words have similar semantics. In summary, Fig. 6 suggests that our code embedding can capture a certain semantics of code words and provide an effective feature space for vulnerability detection.

2) CASE STUDY ON VULNERABILITY ATTENTION

To make our DeepVR Model easier to interpret and to illustrate better what the attention mechanism learns, we visualized the self-attention layer weights. Specifically, we extract the parameters from the trained model's self-attention layer and establish a mapping between the input words and the

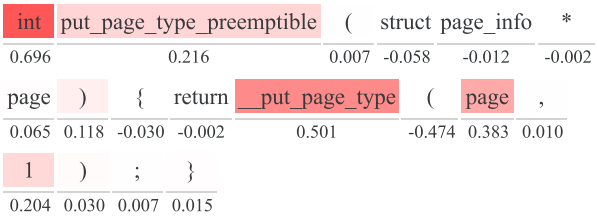


(a) 2D vector space for code embedding



(b) 3D vector space for code embedding

FIGURE 6. 2D vector space for code embedding.



(a) Code attention on the example for vulnerability detection.

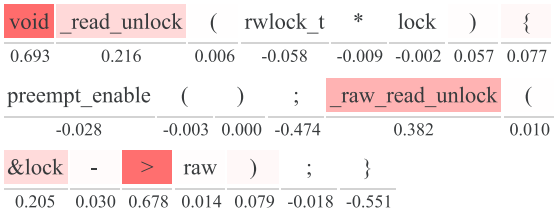
```
 int put_page_type_preemptible(struct page_info *page)
 {
-    return _put_page_type(page, true, NULL);
+    return _put_page_type(page, PTF_preemptible, NULL);
 }
```

(b) Patched code of the example.

FIGURE 7. Code attention on the example for vulnerability detection.

attended weights. We visualize input code words based on their corresponding weights. Different weights refer to various degree of impact on the model decision making. The darker the colour of a code word, the greater its impact on the detection model. Namely, the detection model will pay more attention to the darker word.

In Fig. 7, an code example of CVE-2013-1432 is selected from the open-source dataset to demonstrate a result of vulnerability attention. Fig. 7a shows an detected vulnerability (CVE-2013-1432) by our DeepVR model. This vulnerability is caused by leaking page references which may result in privilege escalation. Fig. 7b shows the *diff* file of the patched vulnerability of CVE-2013-1432. The symbol "+" in front of the statement indicates the added code snippet, and the "-" in front indicates the deleted code snippet. From the *diff* file, we can see that passing one of the parameters to function "__get_page_type" may lead to an impact on the loophole



(a) Attention visualization of vulnerability code.

```
void _read_unlock(rwlock_t *lock)
 {
+    uint32_t x, y;
+
     preempt_enable();
-    _raw_read_unlock(&lock->raw);
+    x = lock->lock;
+    while ( (y = cmpxchg(&lock->lock, x, x-1)) != x )
+        x = y;
 }
```

(b) Modified code for vulnerability code.

FIGURE 8. Attention visualization of vulnerability code.

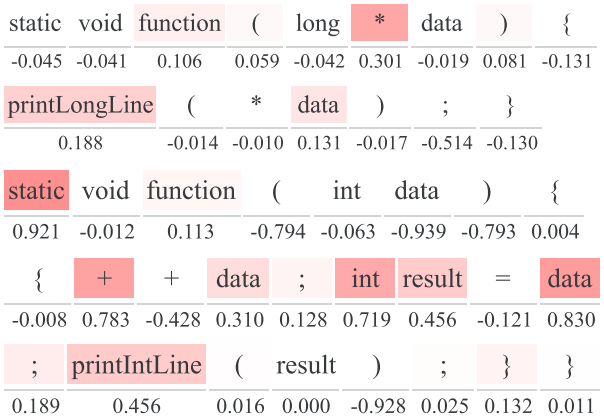


FIGURE 9. Two code examples with attention weights from the SARD dataset.

in the function, which eventually causes the vulnerability. Fig. 7b shows the highlighted keywords based on the attention mechanism. It can be seen that our DeepVR model correctly highlights the vulnerable sink that is the "__get_page_type" function. It has been modified in the *diff* file. The highlighted part can provide clues for code inspectors and guide them to locate the vulnerable code snippet quickly.

In Fig. 8, another code example of CVE-2014-9065 is provided to demonstrate vulnerability attention. Fig. 8a shows a vulnerable code snippet from CVE-2014-9065. According to the CVE report, the code does not handle read-write locks correctly. The *diff* file in Fig. 8b shows the patched code of the vulnerability. They removed the call to the "_ raw_read_unlock()" function and added some code to operate the read-write lock manually. As shown in Fig. 8a, our DeepVR model has highlighted the "_ raw_read_unlock()" function, which is closely related to the vulnerability. This function and the "lock" variable that our DeepVR model has highlighted are consistent with the code that the *diff* file has removed from the vulnerability section. DeepVR can assist code inspectors to the vulnerable code, providing useful clues for locating the vulnerable code snippets.

Fig. 9 shows two examples from the SARD dataset. The parameter of the first function, the character "*", is highlighted by our DeepVR model, which indicates the pointer is important. The character "*" allows one to access the memory address of the "data" variable. This variable is then passed into the highlighted API function "printLongLine", causing the risk of potential memory errors. In the second function, our DeepVR model highlights the variable "data" and the operation on it. Since there is no validation on the size of the variable "data", there may be a risk of out-of-bounds error.

VI. CONCLUSION

Software vulnerability detection is a fundamental problem in cybersecurity. In this article, we proposed a Bi-LSTM model with a self-attention mechanism to address the problem of discovering a bi-directional relationship among some key code to facilitate vulnerability detection. Specifically, the attention mechanism is capable of focusing on the critical part of a code context. With the Bi-LSTM layer placed before the attention layer, the network can also capture the long-range dependencies of the code sequence. Our experiments and results confirmed that our new method performs better than the state-of-the-art deep learning methods. The analysis and visualisation demonstrated that capturing the long-range dependencies of code sequence and identifying the key information of a code context can facilitate the effective learning of potentially vulnerable code patterns.

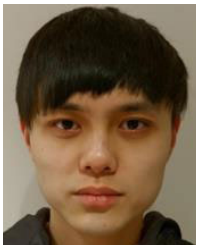
REFERENCES

[1] *6G, the Next Hyper Connected Experience for all*, Issued by Samsung Research, 2020.
[2] J. Corral-Acero *et al.*, "The 'digital twin' to enable the vision of precision cardiology," *Eur. Heart J.*, pp. 1–11, Mar. 2020, doi: 10.1093/eurheartj/ehaa159.
[3] E. J. Topol, "High-performance medicine: The convergence of human and artificial intelligence," *Nature Med.*, vol. 25, no. 1, pp. 44–56, Jan. 2019.
[4] X. Chen, C. Li, D. Wang, S. Wen, J. Zhang, S. Nepal, Y. Xiang, and K. Ren, "Android HIV: A study of repackaging malware for evading machine-learning detection," *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 987–1001, 2020.
[5] L. Liu, O. De Vel, Q.-L. Han, J. Zhang, and Y. Xiang, "Detecting and preventing cyber insider threats: A survey," *IEEE Commun. Surveys Tuts.*, vol. 20, no. 2, pp. 1397–1417, 2nd Quart., 2018.
[6] G. Lin, S. Wen, Q.-L. Han, J. Zhang, and Y. Xiang, "Software vulnerability detection using deep neural networks: A survey," *Proc. IEEE*, vol. 108, no. 10, pp. 1825–1848, Oct. 2020.
[7] N. Sun, J. Zhang, P. Rimba, S. Gao, L. Y. Zhang, and Y. Xiang, "Data-driven cybersecurity incident prediction: A survey," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 2, pp. 1744–1772, 2nd Quart., 2019.
[8] S. Liu, M. Dibaei, Y. Tai, C. Chen, J. Zhang, and Y. Xiang, "Cyber vulnerability intelligence for Internet of Things binary," *IEEE Trans. Ind. Informat.*, vol. 16, no. 3, pp. 2154–2163, Mar. 2020.
[9] G. Lin, W. Xiao, J. Zhang, and Y. Xiang, "Deep learning-based vulnerable function detection: A benchmark," in *Proc. Int. Conf. Inf. Commun. Secur.*, Springer, 2019, pp. 219–232.
[10] R. Coulter, Q.-L. Han, L. Pan, J. Zhang, and Y. Xiang, "Code analysis for intelligent cyber systems: A data-driven approach," *Inf. Sci.*, vol. 524, pp. 46–58, Jul. 2020.

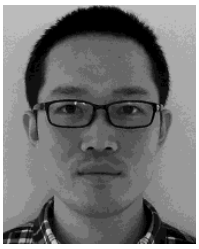
[11] G. Lin, J. Zhang, W. Luo, L. Pan, Y. Xiang, O. De Vel, and P. Montague, "Cross-project transfer representation learning for vulnerable function discovery," *IEEE Trans. Ind. Informat.*, vol. 14, no. 7, pp. 3289–3297, Jul. 2018.
[12] R. Coulter, Q.-L. Han, L. Pan, J. Zhang, and Y. Xiang, "Data-driven cyber security in perspective—Intelligent traffic analysis," *IEEE Trans. Cybern.*, vol. 50, no. 7, pp. 3081–3093, Jul. 2020.
[13] S. Liu, G. Lin, Q.-L. Han, S. Wen, J. Zhang, and Y. Xiang, "DeepBalance: Deep-learning and fuzzy oversampling for vulnerability detection," *IEEE Trans. Fuzzy Syst.*, vol. 28, no. 7, pp. 1329–1343, Jul. 2020.
[14] M. Wang, T. Zhu, T. Zhang, J. Zhang, S. Yu, and W. Zhou, "Security and privacy in 6G networks: New areas and new challenges," *Digit. Commun. Netw.*, vol. 6, no. 3, pp. 281–291, Aug. 2020.
[15] S. Liu, G. Lin, L. Qu, J. Zhang, O. De Vel, P. Montague, and Y. Xiang, "CD-VulD: Cross-domain vulnerability discovery based on deep domain adaptation," *IEEE Trans. Depend. Sec. Comput.*, early access, Apr. 2, 2020, doi: 10.1109/TDSC.2020.2984505.
[16] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," *Neural Netw.*, vol. 2, no. 5, pp. 359–366, Jan. 1989.
[17] Y. Lee, H. Kwon, S.-H. Choi, S.-H. Lim, S. H. Baek, and K.-W. Park, "instruction2vec: Efficient preprocessor of assembly code to detect software weakness with CNN," *Appl. Sci.*, vol. 9, no. 19, p. 4086, Sep. 2019.
[18] F. Wu, J. Wang, J. Liu, and W. Wang, "Vulnerability detection with deep learning," in *Proc. 3rd IEEE Int. Conf. Comput. Commun. (ICCC)*, Dec. 2017, pp. 1298–1302.
[19] G. Berikol, S. Kula, and O. Yildiz, "Machine learning techniques in diagnosis of pulmonary embolism," *J. Clin. Anal. Med.*, vol. 6, pp. 729–732, Dec. 2015.
[20] V. Cukic and A. Ustamujic, "Lung cancer and pulmonary thromboembolism," *Materia Socio-Medica*, vol. 27, no. 5, pp. 351–353, 2015.
[21] C. Bellinger, M. S. Mohomed Jabbar, O. Zaïane, and A. Osornio-Vargas, "A systematic review of data mining and machine learning for air pollution epidemiology," *BMC Public Health*, vol. 17, no. 1, p. 907, Dec. 2017.
[22] L. Agharezaei, Z. Agharezaei, A. Nemati, K. Bahaadinbeigy, F. Keynia, M. Baneshi, A. Iranpour, and M. Agharezaei, "The prediction of the risk level of pulmonary embolism and deep vein thrombosis through artificial neural network," *Acta Inf. Medica*, vol. 24, no. 5, pp. 354–359, 2016.
[23] I. Banerjee, M. Sofela, J. Yang, J. H. Chen, N. H. Shah, R. Ball, A. I. Mushlin, M. Desai, J. Bledsoe, T. Amrhein, and D. L. Rubin, "Development and performance of the pulmonary embolism result forecast model (perform) for computed tomography clinical decision support," *JAMA Netw. Open*, vol. 2, no. 8, pp. 1760–1768, 2019.
[24] P. Rajpurkar, J. Irvin, K. Zhu, B. Yang, H. Mehta, T. Duan, D. Ding, A. Bagul, C. Langlotz, K. Shpanskaya, M. P. Lungren, and A. Y. Ng, "CheXNet: Radiologist-level pneumonia detection on chest X-rays with deep learning," 2017, *arXiv:1711.05225*. [Online]. Available: http://arxiv.org/abs/1711.05225
[25] W. Zhou, Y. Jia, A. Peng, Y. Zhang, and P. Liu, "The effect of IoT new features on security and privacy: New threats, existing solutions, and challenges yet to be solved," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 1606–1616, Apr. 2019.
[26] S. Vicini, S. Bellini, A. Rosi, and A. Sanna, "Well-being on the go: An iot vending machine service for the promotion of healthy behaviors and lifestyles," in *Proc. Int. Conf. Design, User Exper., Usability*, 2013, pp. 594–603.
[27] K. Fu, T. Kohno, D. Lopresti, E. Mynatt, K. Nahrstedt, S. Patel, D. Richardson, and B. Zorn, "Safety, security, and privacy threats posed by accelerating trends in the Internet of Things," *Comput. Community Consortium (CCC) Tech. Rep.*, vol. 29, no. 3, pp. 1–9, 2017.
[28] Y. Yang, L. Wu, G. Yin, L. Li, and H. Zhao, "A survey on security and privacy issues in Internet-of-Things," *IEEE Internet Things J.*, vol. 4, no. 5, pp. 1250–1258, Oct. 2017.
[29] A. Boejen and C. Grau, "Virtual reality in radiation therapy training," *Surgical Oncol.*, vol. 20, no. 3, pp. 185–188, Sep. 2011.
[30] S. C. Sethuraman, V. Vijayakumar, and S. Walczak, "Cyber attacks on healthcare devices using unmanned aerial vehicles," *J. Med. Syst.*, vol. 44, no. 1, p. 29, Jan. 2020.
[31] F. Mohamed, J. Abdeslam, and E. B. Lahcen, "Towards new approach to enhance learning based on Internet of Things and virtual reality," in *Proc. Int. Conf. Learn. Optim. Algorithms, Theory Appl. LOPAL*, 2018, pp. 1–5.
[32] C. G. Coogan and B. He, "Brain-computer interface control in a virtual reality environment and applications for the Internet of Things," *IEEE Access*, vol. 6, pp. 10840–10849, 2018.
[33] B. Ramsundar and R. B. Zadeh, *TensorFlow for Deep Learning: From Linear Regression to Reinforcement Learning*. Newton, MA, USA: O'Reilly Media, 2018.
[34] L. K. Shar and H. B. K. Tan, "Predicting common Web application vulnerabilities from input validation and sanitization code patterns," in *Proc. 27th IEEE/ACM Int. Conf. Automated Softw. Eng. ASE*, Sep. 2012, pp. 310–313.
[35] G. Grieco, G. L. Grinblat, L. Uzal, S. Rawat, J. Feist, and L. Mounier, "Toward large-scale vulnerability discovery using machine learning," in *Proc. 6th ACM Conf. Data Appl. Secur. Privacy - CODASPY*, 2016, pp. 85–96.
[36] F. Dong, J. Wang, Q. Li, G. Xu, and S. Zhang, "Defect prediction in Android binary executables using deep neural network," *Wireless Pers. Commun.*, vol. 102, no. 3, pp. 2261–2285, Oct. 2018.
[37] H. Peng, L. Mou, G. Li, Y. Liu, L. Zhang, and Z. Jin, "Building program vector representations for deep learning," in *Proc. Int. Conf. Knowl. Sci., Eng. Manage.*, Springer, 2015, pp. 547–553.
[38] I. V. Krsul, *Software Vulnerability Analysis*. West Lafayette, IN, USA: Purdue University West Lafayette, 1998.
[39] Y. J. Lee, S.-H. Choi, C. Kim, S.-H. Lim, and K.-W. Park, "Learning binary code with deep learning to detect software weakness," in *Proc. KSII The 9th Int. Conf. Internet (ICONI) 2017 Symp.*, 2017.
[40] R. Russell, L. Kim, L. Hamilton, T. Lazovich, J. Harer, O. Ozdemir, P. Ellingwood, and M. McConley, "Automated vulnerability detection in source code using deep representation learning," in *Proc. 17th IEEE Int. Conf. Mach. Learn. Appl. (ICMLA)*, Dec. 2018, pp. 757–762.
[41] Z. Li, D. Zou, S. Xu, H. Jin, H. Qi, and J. Hu, "VulPecker: An automated vulnerability detection system based on code similarity analysis," in *Proc. 32nd Annu. Conf. Comput. Secur. Appl.*, Dec. 2016, pp. 201–213.
[42] G. Lin, J. Zhang, W. Luo, L. Pan, and Y. Xiang, "POSTER: Vulnerability discovery with function representation learning from unlabeled projects," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2017, pp. 2539–2541.
[43] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, and Z. Chen, "SySeVR: A framework for using deep learning to detect software vulnerabilities," 2018, *arXiv:1807.06756*. [Online]. Available: http://arxiv.org/abs/1807.06756
[44] G. Lin, J. Zhang, W. Luo, L. Pan, O. De Vel, P. Montague, and Y. Xiang, "Software vulnerability discovery via learning multi-domain knowledge bases," *IEEE Trans. Depend. Sec. Comput.*, early access, Nov. 19, 2020, doi: 10.1109/TDSC.2019.2954088.
[45] (2018). *Software Assurance Reference Dataset Project*. Accessed: May 20, 2018. Online]. Available: https://samate.nist.gov/SRD/
[46] M. Allamanis, E. T. Barr, P. Devanbu, and C. Sutton, "A survey of machine learning for big code and naturalness," *ACM Comput. Surv.*, vol. 51, no. 4, p. 81, Jul. 2018.
[47] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," in *Proc. Adv. Neural Inf. Process. Syst.*, 2013, pp. 3111–3119.
[48] C. Olah, "Understanding LSTM networks," *GITHUB Blog*, to be published.
[49] J. Wang, X. Peng, and Y. Qiao, "Cascade multi-head attention networks for action recognition," *Comput. Vis. Image Understand.*, vol. 192, Mar. 2020, Art. no. 102898.
[50] L. Ye, M. Rochan, Z. Liu, and Y. Wang, "Cross-modal self-attention network for referring image segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 10502–10511.
[51] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 5998–6008.
[52] Z. Lin, M. Feng, C. Nogueira dos Santos, M. Yu, B. Xiang, B. Zhou, and Y. Bengio, "A structured self-attentive sentence embedding," 2017, *arXiv:1703.03130*. [Online]. Available: http://arxiv.org/abs/1703.03130
[53] X. Sun and W. Lu, "Understanding attention for text classification," in *Proc. 58th Annu. Meeting Assoc. Comput. Linguistics*, 2020, pp. 3418–3428.
[54] F. Chollet *et al.* (2015). *Keras*. [Online]. Available: https://github.com/fchollet/keras
[55] H. Schütze, C. D. Manning, and P. Raghavan, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge Univ. Press, 2008, vol. 39.
[56] D. A. Wheeler. (2016). *Flawfinder*. Accessed: May 20, 2016. [Online]. Available: https://www.dwheeler.com/flawfinder/

[57] H. Perl, S. Dechand, M. Smith, D. Arp, F. Yamaguchi, K. Rieck, S. Fahl, and Y. Acar, "VCCFinder: Finding potential vulnerabilities in open-source projects to assist code audits," in *Proc. 22nd ACM SIGSAC Conf. Comput. Commun. Secur. - CCS*, 2015, pp. 426–437.
[58] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, and Y. Zhong, "VulDeePecker: A deep learning-based system for vulnerability detection," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2018, pp. 1–15.
[59] A. Karpathy, J. Johnson, and L. Fei-Fei, "Visualizing and understanding recurrent networks," 2015, *arXiv:1506.02078*. [Online]. Available: http://arxiv.org/abs/1506.02078

JUN ZHANG is currently a Professor with the School of Physics and Electronic Information, Yunnan Normal University, China. He has published more than 100 research papers in refereed international journals and conferences. His research interests include applied AI, big data analytics, precision medicine, and healthcare digital twin. He has served as the Chair in many international conferences.



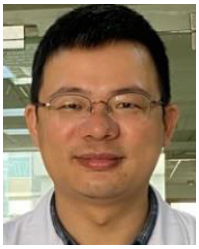
LIN LI is currently a Visiting Research Student with the School of Physics and Electronic Information, Yunnan Normal University, China. His research interests include applied AI, cyber resilience, and healthcare digital twin.



GUANJUN LIN received the bachelor's degree in information technology (Hons.) from Deakin University, Geelong, VIC, Australia, in 2012. He is currently pursuing the Ph.D. degree with the School of Software and Electrical Engineering, Swinburne University of Technology, Melbourne, VIC, Australia. His research interest includes the application of deep learning techniques for software vulnerability detection.

DA FANG received the master's degree in optic engineering from Yunnan Normal University, Kunming, China, in 2012, where he is currently pursuing the Ph.D. degree with the School of Physics and Electronic Information. His current research interests include physics-based rendering, medical image processing, and optoelectronic information technology.

YONGHANG TAI received the Ph.D. degree in computer science from IISRI, Deakin University, Geelong, VIC, Australia, in 2019. He is currently a Professor with the School of Physics and Electronic Information, Yunnan Normal University, China. His current research interests include physics-based simulation, applied AI, medical AR/MR, and precision medicine.



JIECHUN HUANG received the Doctor of Cardiothoracic Surgery degree from Fudan University. He is currently a Clinician with the Cardiothoracic Surgery Department, Huashan Hospital, Fudan University. His research interests include surgical treatment of lung cancer, surgical treatment of coronary heart disease, and mechanisms of myocardial apoptosis.